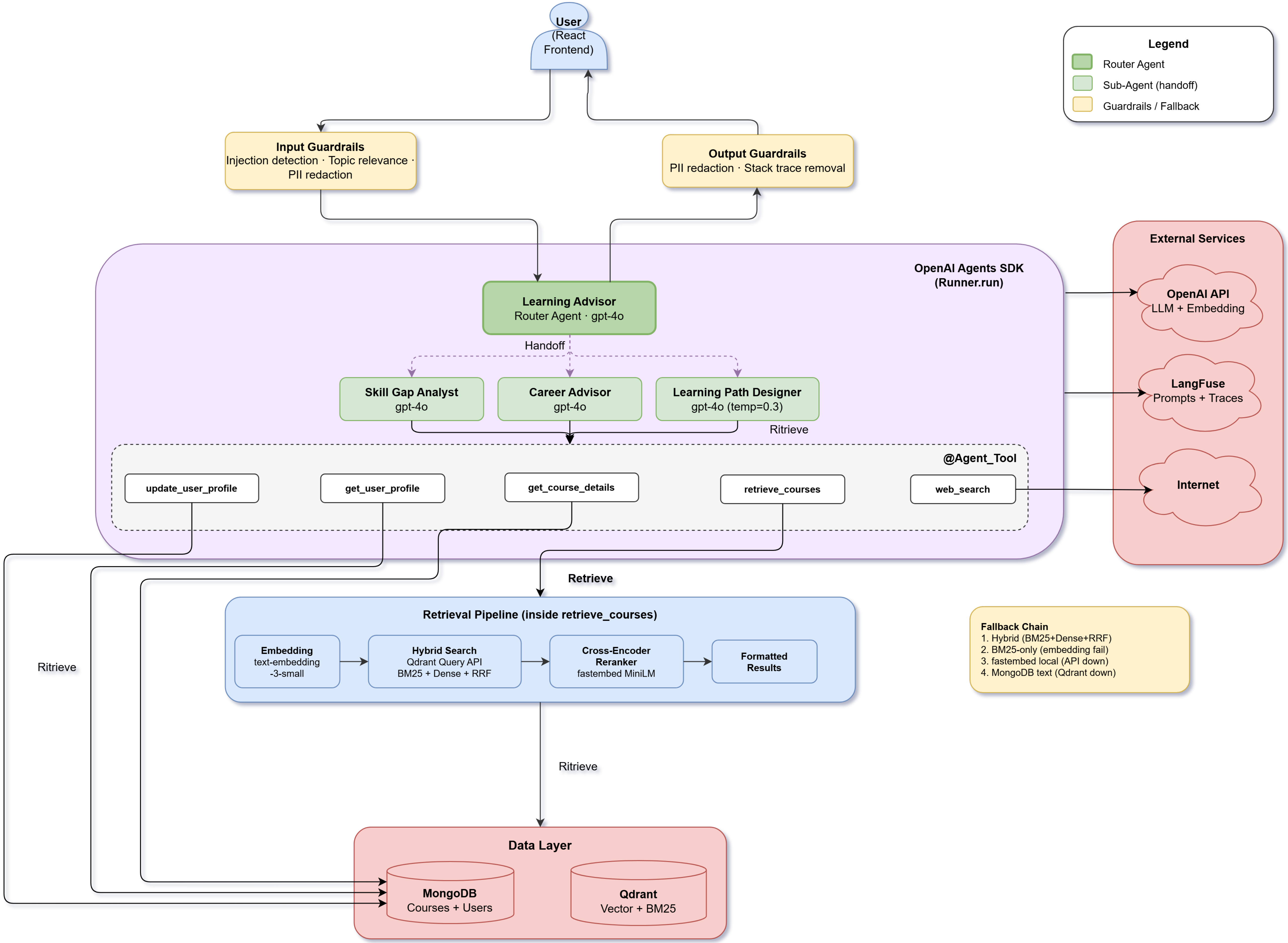
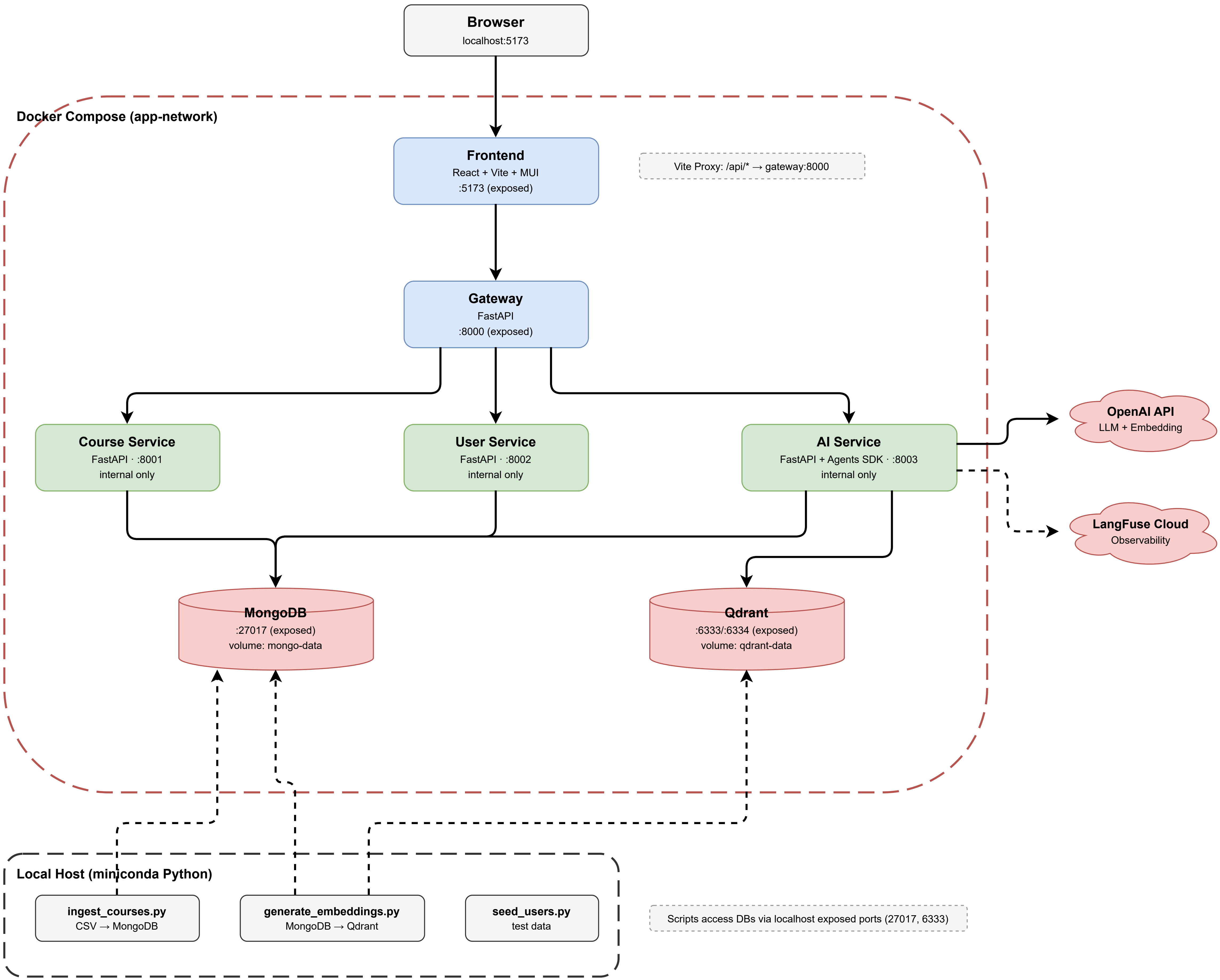
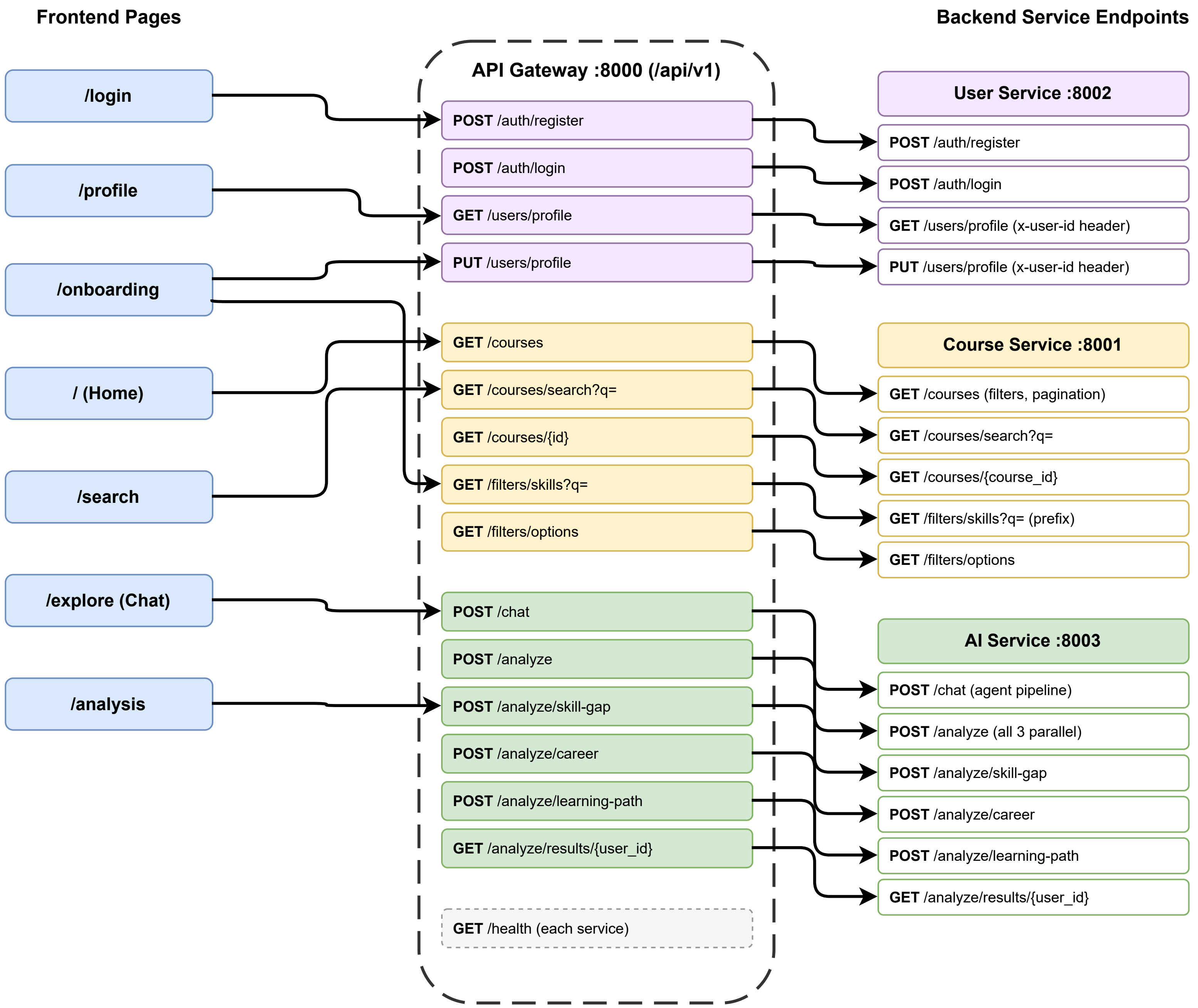
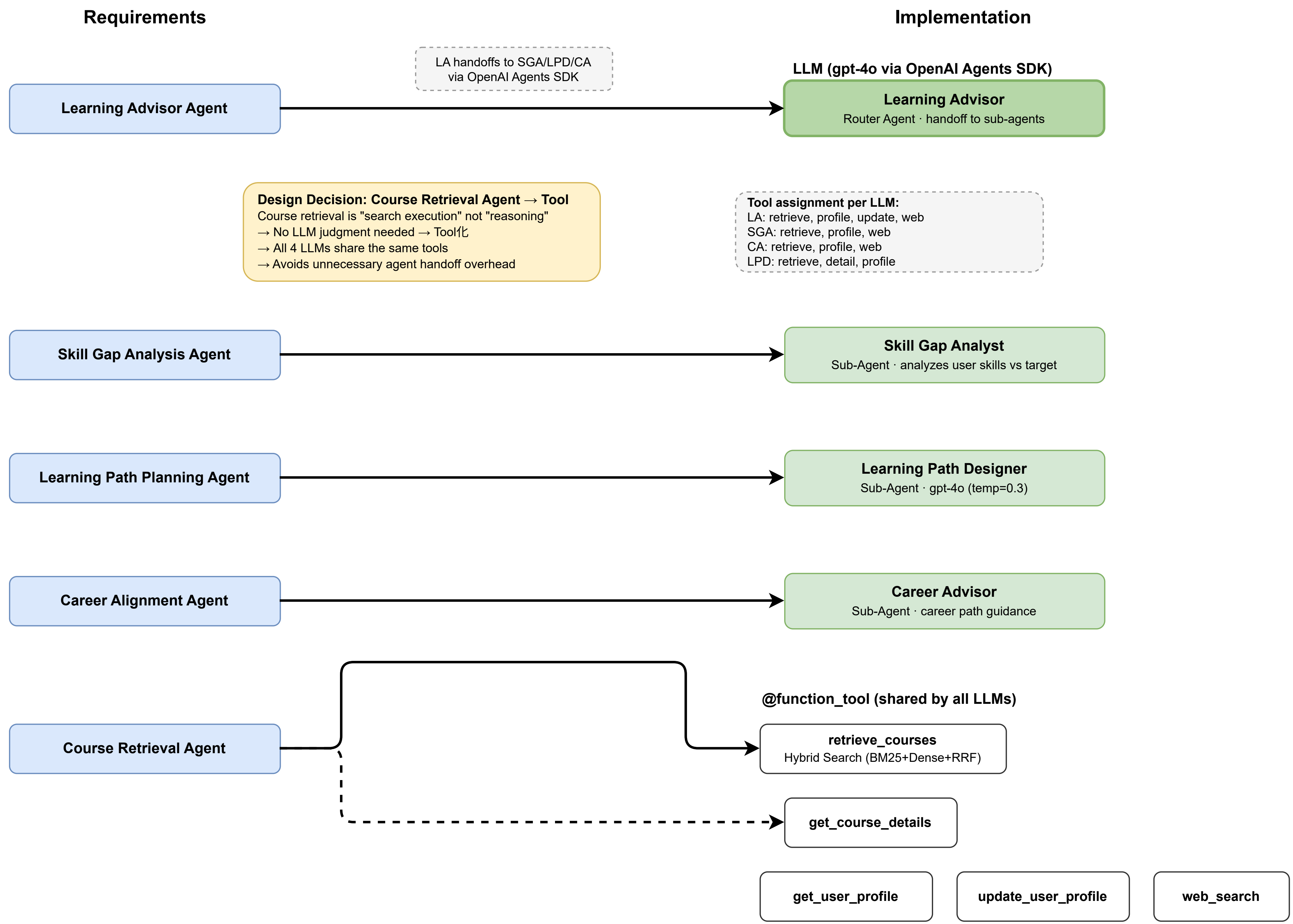

User
(React
Frontend)
Legend
Router Agent
Sub-Agent (handoff)
Guardrails / Fallback
Input Guardrails
Injection detection · Topic relevance ·
PII redaction
Output Guardrails
PII redaction · Stack trace removal
External Services
OpenAI Agents SDK
(Runner.run)
Learning Advisor
Router Agent · gpt-4o
OpenAI API
LLM + Embedding
Handoff
Skill Gap Analyst
gpt-4o
Career Advisor
gpt-4o
Learning Path Designer
gpt-4o (temp=0.3)
LangFuse
Prompts + Traces
Ritrieve
@Agent_Tool
update_user_profile
get_user_profile
get_course_details
retrieve_courses
web_search
Internet
Retrieve
Retrieval Pipeline (inside retrieve_courses)
Embedding
text-embedding
-3-small
Hybrid Search
Qdrant Query API
BM25 + Dense + RRF
Cross-Encoder
Reranker
fastembed MiniLM
Formatted
Results
Fallback Chain
1. Hybrid (BM25+Dense+RRF)
2. BM25-only (embedding fail)
3. fastembed local (API down)
4. MongoDB text (Qdrant down)
Ritrieve
Ritrieve
Data Layer
MongoDB
Courses + Users
Qdrant
Vector + BM25

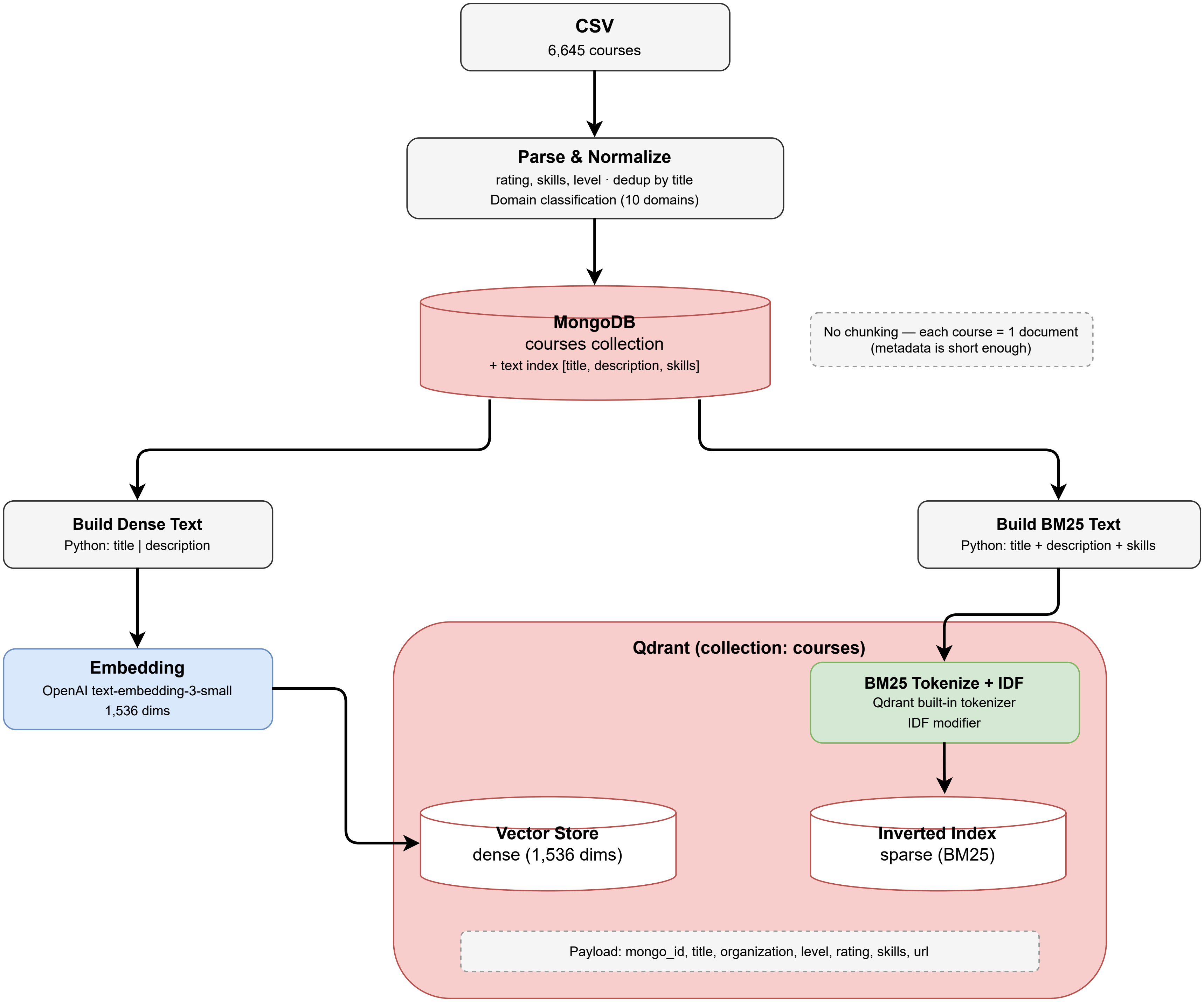


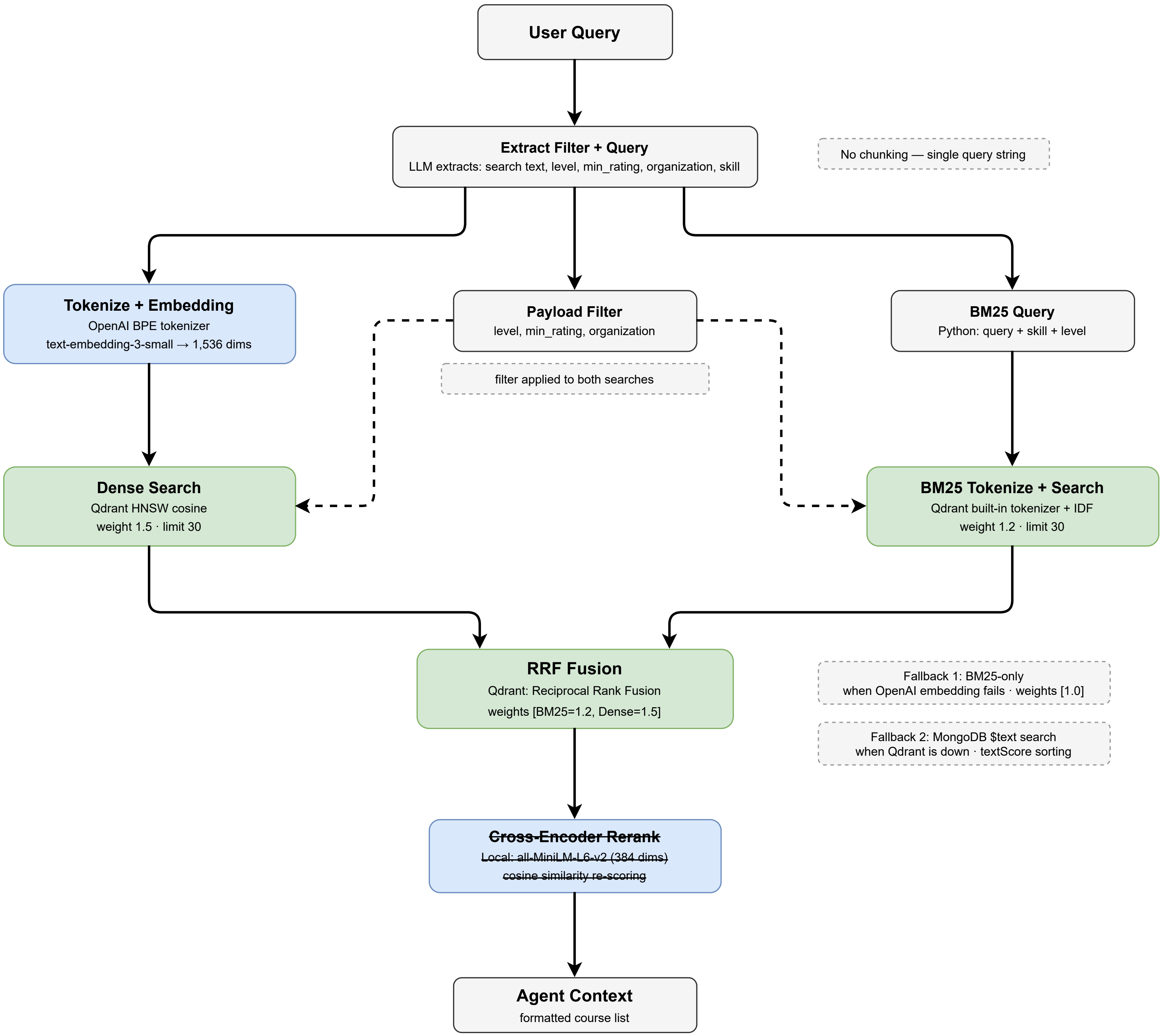
Browser
localhost:5173
Docker Compose (app-network)
Frontend
React + Vite + MUI
:5173 (exposed)
Vite Proxy: /api/* → gateway:8000
Gateway
FastAPI
:8000 (exposed)
Course Service
FastAPI · :8001
internal only
User Service
FastAPI · :8002
internal only
AI Service
FastAPI + Agents SDK · :8003
internal only
OpenAI API
LLM + Embedding
LangFuse Cloud
Observability
MongoDB
:27017 (exposed)
volume: mongo-data
Qdrant
:6333/:6334 (exposed)
volume: qdrant-data
Local Host (miniconda Python)
ingest_courses.py
CSV → MongoDB
generate_embeddings.py
MongoDB → Qdrant
seed_users.py
test data
Scripts access DBs via localhost exposed ports (27017, 6333)

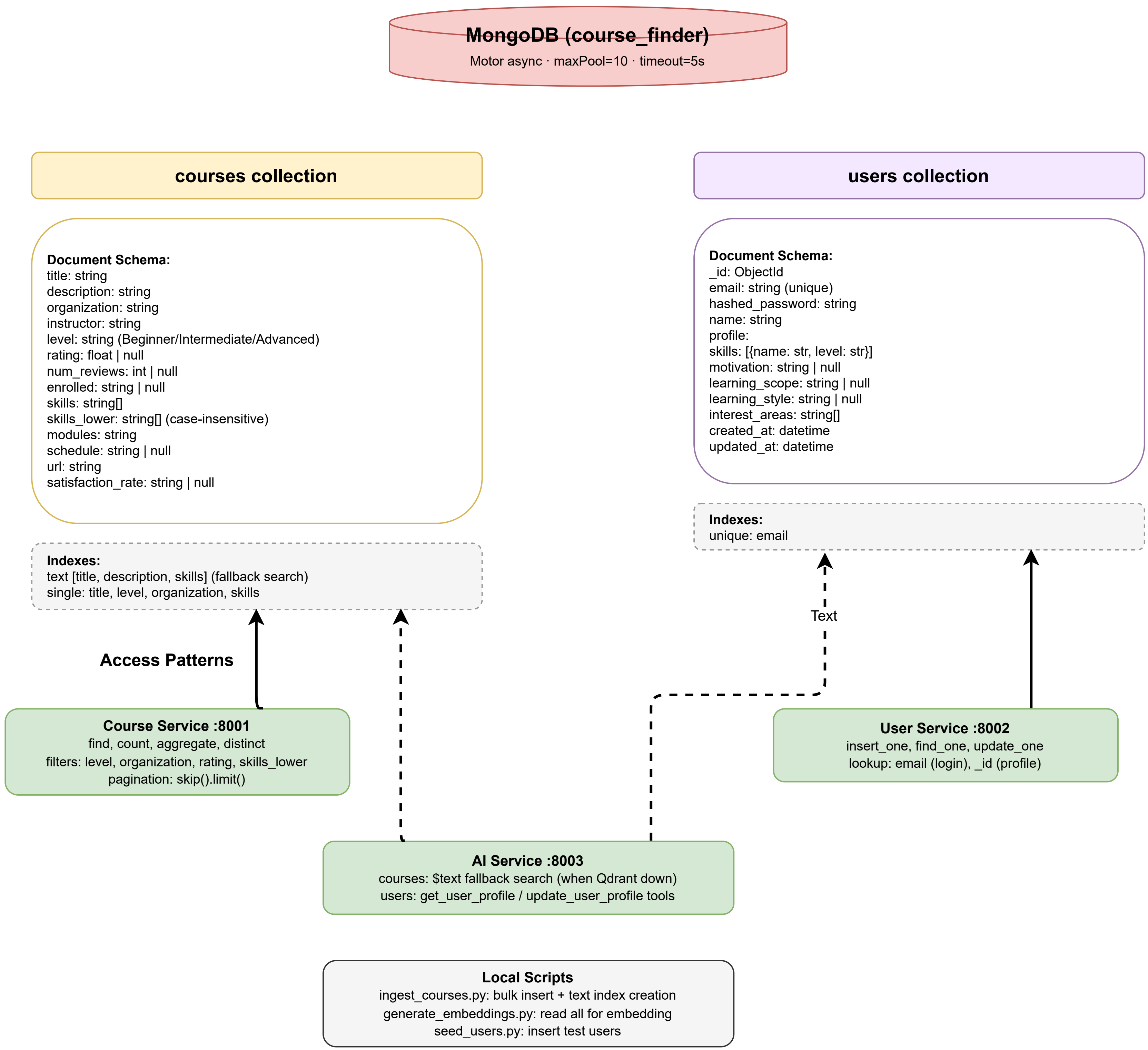


Frontend Pages
Backend Service Endpoints
/login
/profile
/onboarding
/ (Home)
/search
/explore (Chat)
/analysis
API Gateway :8000 (/api/v1)
POST /auth/register
POST /auth/login
GET /users/profile
PUT /users/profile
GET /courses
GET /courses/search?q=
GET /courses/{id}
GET /filters/skills?q=
GET /filters/options
POST /chat
POST /analyze
POST /analyze/skill-gap
POST /analyze/career
POST /analyze/learning-path
GET /analyze/results/{user_id}
GET /health (each service)
User Service :8002
POST /auth/register
POST /auth/login
GET /users/profile (x-user-id header)
PUT /users/profile (x-user-id header)
Course Service :8001
GET /courses (filters, pagination)
GET /courses/search?q=
GET /courses/{course_id}
GET /filters/skills?q= (prefix)
GET /filters/options
AI Service :8003
POST /chat (agent pipeline)
POST /analyze (all 3 parallel)
POST /analyze/skill-gap
POST /analyze/career
POST /analyze/learning-path
GET /analyze/results/{user_id}


Requirements
Implementation
LA handoffs to SGA/LPD/CA
via OpenAI Agents SDK
LLM (gpt-4o via OpenAI Agents SDK)
Learning Advisor Agent
Learning Advisor
Router Agent · handoff to sub-agents
Design Decision: Course Retrieval Agent → Tool
Course retrieval is "search execution" not "reasoning"
→ No LLM judgment needed → Tool化
→ All 4 LLMs share the same tools
→ Avoids unnecessary agent handoff overhead
Tool assignment per LLM:
LA: retrieve, profile, update, web
SGA: retrieve, profile, web
CA: retrieve, profile, web
LPD: retrieve, detail, profile
Skill Gap Analysis Agent
Skill Gap Analyst
Sub-Agent · analyzes user skills vs target
Learning Path Planning Agent
Learning Path Designer
Sub-Agent · gpt-4o (temp=0.3)
Career Alignment Agent
Career Advisor
Sub-Agent · career path guidance
@function_tool (shared by all LLMs)
Course Retrieval Agent
retrieve_courses
Hybrid Search (BM25+Dense+RRF)
get_course_details
get_user_profile
update_user_profile
web_search


CSV
6,645 courses
Parse & Normalize
rating, skills, level · dedup by title
Domain classification (10 domains)
MongoDB
courses collection
+ text index [title, description, skills]
No chunking — each course = 1 document
(metadata is short enough)
Build Dense Text
Python: title | description
Build BM25 Text
Python: title + description + skills
Embedding
OpenAI text-embedding-3-small
1,536 dims
Qdrant (collection: courses)
BM25 Tokenize + IDF
Qdrant built-in tokenizer
IDF modifier
Vector Store
dense (1,536 dims)
Inverted Index
sparse (BM25)
Payload: mongo_id, title, organization, level, rating, skills, url


User Query
Extract Filter + Query
LLM extracts: search text, level, min_rating, organization, skill
No chunking — single query string
Tokenize + Embedding
OpenAI BPE tokenizer
text-embedding-3-small → 1,536 dims
Payload Filter
level, min_rating, organization
filter applied to both searches
BM25 Query
Python: query + skill + level
Dense Search
Qdrant HNSW cosine
weight 1.5 · limit 30
BM25 Tokenize + Search
Qdrant built-in tokenizer + IDF
weight 1.2 · limit 30
RRF Fusion
Qdrant: Reciprocal Rank Fusion
weights [BM25=1.2, Dense=1.5]
Fallback 1: BM25-only
when OpenAI embedding fails · weights [1.0]
Fallback 2: MongoDB $text search
when Qdrant is down · textScore sorting
~~Cross-Encoder Rerank~~
~~Local: all-MiniLM-L6-v2 (384 dims)~~
~~cosine similarity re-scoring~~
Agent Context
formatted course list


MongoDB (course_finder)
Motor async · maxPool=10 · timeout=5s
courses collection
Document Schema:
title: string
description: string
organization: string
instructor: string
level: string (Beginner/Intermediate/Advanced)
rating: float | null
num_reviews: int | null
enrolled: string | null
skills: string[]
skills_lower: string[] (case-insensitive)
modules: string
schedule: string | null
url: string
satisfaction_rate: string | null
Indexes:
text [title, description, skills] (fallback search)
single: title, level, organization, skills
users collection
Document Schema:
_id: ObjectId
email: string (unique)
hashed_password: string
name: string
profile:
skills: [{name: str, level: str}]
motivation: string | null
learning_scope: string | null
learning_style: string | null
interest_areas: string[]
created_at: datetime
updated_at: datetime
Indexes:
unique: email
Text
Access Patterns
Course Service :8001
find, count, aggregate, distinct
filters: level, organization, rating, skills_lower
pagination: skip().limit()
User Service :8002
insert_one, find_one, update_one
lookup: email (login), _id (profile)
AI Service :8003
courses: $text fallback search (when Qdrant down)
users: get_user_profile / update_user_profile tools
Local Scripts
ingest_courses.py: bulk insert + text index creation
generate_embeddings.py: read all for embedding
seed_users.py: insert test users